VIETNAM-KOREA UNIVERSITY OF INFORMATION AND COMMUNICATION TECHNOLOGY (VKU)
FACULTY OF COMPUTER SCIENCE – CAPSTONE PROJECT 2 (DACN2) PRELIMINARY REPORT

An Autonomous Cyber-Defense Framework Using Golang Ingress Telemetry and Machine Learning for Distinguishing Layer-7 DDoS from Flash Crowds on Kubernetes

Quoc Thien Le[1], Quang Huy Nguyen[1], Ngoc Bao Vo[1], and Dr. Ngoc Tho Huynh[1*]

[1]Faculty of Computer Science, Vietnam-Korea University of Information and Communication Technology, The University of Danang
Student Team: Quoc Thien Le (23IT.B209), Quang Huy Nguyen (23IT105), Ngoc Bao Vo (23IT.B010)

[*]Project Advisor: Dr. Ngoc Tho Huynh – Contact: thienlq.23itb@vku.udn.vn

ABSTRACT

High-concurrency web services often suffer sudden traffic surges caused either by authentic users (*Flash Crowds*) or by malicious application-layer attacks (*Layer-7 DDoS*). Distinguishing between these two phenomena is a critical challenge. Standard firewalls and CPU-based autoscalers cannot differentiate authentic requests from bot floods because both establish valid TCP/TLS connections. Relying on external Content Delivery Networks (CDNs) introduces a fundamental blind spot: terminating TLS at the edge strips raw client TLS fingerprints and hides static asset requests from the origin server. Furthermore, when attackers launch a DDoS attack during a legitimate flash event (*Mixed Traffic*), traditional binary classifiers either crash or induce Economic Denial of Sustainability (EDoS) by over-allocating cloud servers to serve attacking bots. This preliminary capstone report presents an autonomous, domain-agnostic cyber-defense framework that eliminates external CDN dependencies. The architecture deploys a custom Golang Ingress Gateway that terminates TLS directly, extracts raw JA4 TLS fingerprints and HTTP/2 frame signatures in memory, and computes real-time per-client suspicion scores. Every 5 seconds, the gateway forwards 12 cross-layer statistical features to a Python AI Control Plane. A four-class machine learning model classifies traffic into `Normal`, `Flash_Crowd`, `Pure_L7_DDoS`, or `Mixed_Traffic`. When attacks occur, the gateway dynamically drops hostile connections, while the Kubernetes (K3s) cluster scales backend pods strictly according to the admitted legitimate request rate. We present the system architecture, mathematical formulation, feature engineering, and a rigorous training protocol using GroupKFold and in-fold SMOTE to achieve F1 $> 95\%$ and FPR $\leq 2\%$.

Keywords: Flash Crowd, Layer-7 DDoS, Golang Ingress Gateway, JA4 TLS Fingerprinting, Machine Learning, Anti-EDoS Auto-Scaling, Kubernetes (K3s), Capstone Project.

1 INTRODUCTION

Modern web applications—such as university course registration portals, online ticketing platforms, and e-commerce stores—regularly encounter sharp, unexpected spikes in incoming traffic. These spikes generally stem from two opposing phenomena:

1. **Legitimate Flash Crowds (FC):** Thousands of real human users access the service simultaneously for a scheduled event (e.g., student course enrollment or limited product sales).
2. **Layer-7 (L7) DDoS Attacks:** Distributed botnets flood the application with syntactically valid HTTP/HTTPS requests (such as Challenge Collapsar cache-busting or HTTP/2 rapid reset floods) designed to exhaust backend CPU, database connections, and memory.

Traditional Layer-3/4 firewalls cannot distinguish between these two events because both originate from valid TCP handshakes and transmit proper HTTP headers. When faced with traffic surges, Kubernetes Horizontal Pod Autoscalers (HPA) typically scale server pods based solely on CPU or memory thresholds. During an L7 DDoS flood, this naive scaling spawns surplus pods to serve malicious traffic, draining cloud computing budgets without resolving the bottleneck—a catastrophic outcome known as *Economic Denial of Sustainability (EDoS)*.

1.1 Why External CDNs Fall Short

Many web systems place a third-party Content Delivery Network (CDN) in front of their servers for protection. However, this introduces serious architectural drawbacks for fine-grained anomaly detection:

- **TLS Fingerprint Blind Spot:** CDNs terminate client TLS connections at their edge servers. The origin server only sees the CDN's own TLS stack, completely losing the client's original TLS `ClientHello` extensions and true JA4 fingerprint.
- **Static Asset Masking:** Edge caching serves images, CSS, and JavaScript files directly from CDN edge caches. As a result, the origin server cannot track the ratio of static-to-dynamic requests, blinding behavioral analysis.
- **The Mixed-Traffic Dilemma:** When attackers launch a DDoS attack right in the middle of a flash crowd, CDN challenge pages (e.g., CAPTCHAs) degrade the experience of authentic users, while rate limiters drop requests indiscriminately.

1.2 Proposed Sovereign Solution and KPIs

To overcome these limitations, this Capstone Project (DACN2) proposes an autonomous, sovereign cyber-defense framework that eliminates external CDN dependencies. We deploy a lightweight, custom **Golang Security Ingress Gateway** directly at the infrastructure perimeter. The gateway terminates TLS, inspects raw network packets to extract cryptographic fingerprints, scores individual clients, and coordinates with an AI engine to scale backend pods safely.

The project targets four concrete engineering KPIs:

- Multi-class classification accuracy: Macro F1-score $> 95\%$ across all four operational states, with False Positive Rate FPR $\leq 2\%$ on authentic user traffic.
- Real-time AI inference latency below 10 ms per 5-second analysis window.
- Elastic auto-scaling on K3s from 1 to 5 pods within 4 seconds during authentic flash crowds.
- Immediate mitigation during L7 DDoS attacks: dropping hostile sockets at the gateway and scaling pods strictly by admitted legitimate requests to eliminate EDoS.

2 RELATED WORKS

Differentiating flash crowds from malicious L7 DDoS attacks has been explored through several approaches:

- **Statistical and Entropy Thresholding:** Early studies analyzed source IP address distribution and Shannon entropy [10]. David and Thomas [9] developed dynamic thresholding on request rate variations. However, static entropy thresholds frequently trigger false alarms during flash crowds because authentic users also concentrate on a small set of popular pages.
- **Transport-Layer Flow Modeling:** Abdulkadir et al. (2025) [1] applied Random Forest classifiers with SMOTE

on the CICIDS 2017 benchmark, relying on flow duration and byte rates. While effective for Layer-3/4 volumetric attacks, these metrics are blind to application-layer semantics and HTTP/2 multiplexing anomalies.

- **Application Log Analysis:** Biruk [8] and Muse and Abebe [4] examined web server access logs using historical traces (such as the 1998 FIFA World Cup). Althobaiti et al. [5] designed an ensemble intrusion detection system for cloud environments. Tinubu et al. [6] combined a multi-layer perceptron with request queuing to absorb flash surges. However, log-parsing approaches operate after requests have already entered the server, introducing detection lag.
- **TLS and JA4 Fingerprinting:** FoxIO introduced the JA4+ suite [2], which hashes TLS cipher suites and extension lists while neutralizing RFC 8701 GREASE randomization. Recent research [3] demonstrated that combining JA4 hashes with HTTP/2 frame signatures separates automated script toolkits from genuine web browsers with high precision.

Research Gap: Existing studies either rely on transport-layer metrics, depend on post-mortem log files, or treat detection as a simple binary choice. None provide an integrated, real-time architecture that handles mixed traffic (DDoS overlapping with a flash crowd) while safely guiding container autoscaling.

3 PROPOSED SYSTEM ARCHITECTURE

3.1 Three-Tier Sovereign System Topology

As shown in Figure 1, the proposed defense framework operates across three tightly integrated tiers without external third-party dependencies:

- **Tier 1 (Golang Security Ingress Gateway):** The public-facing entry point. It terminates TLS connections directly using locally trusted certificates (`mkcert` for lab environments) or automated ACME certificates (`autocert` for public domains). The gateway inspects raw `ClientHello` packets to extract true JA4 hashes, taps HTTP/2 frame streams, calculates per-client suspicion scores, drops hostile sockets, and exports telemetry every 5 seconds.
- **Tier 2 (AI Control Plane & Orchestrator):** An asynchronous Python FastAPI service. It normalizes incoming telemetry, executes four-class machine learning inference, pushes dynamic filtering thresholds back to the gateway, and calculates the exact pod replicas required for K3s. Communication between the orchestrator and gateway is authenticated via a shared secret API token.
- **Tier 3 (Elastic Application Backend):** A containerized web application (NestJS connected to PostgreSQL) deployed on a lightweight Kubernetes (K3s) cluster. Each pod is restricted by strict cgroup micro-quotas (0.2 vCPU, 256MB RAM) to benchmark elastic horizontal scaling.

3.2 Standard TLS/HTTP2 Fingerprints and Per-Client Scoring

The gateway uses the published FoxIO JA4 algorithm [2] and the Akamai HTTP/2 signature format [13], with BrowserLeaks-compatible output fields [14]. On the selected Go toolchain, `ClientHelloInfo` provides the required ordered inputs; no fixed-size TCP peek is required. Standard JA4 sorts cipher and extension lists before hashing, while the FoxIO original-order variants preserve their order:

1. The `GetConfigForClient` callback reads cipher suites, extensions, supported versions, signature algorithms and ALPN without reading from the socket. It emits `ja4`, `ja4_r`, `ja4_o`, and `ja4_ro`, excluding GREASE according to FoxIO.
2. After TLS decryption, a bounded passive observer copies frame bytes while Go remains the protocol owner. The first request's `HEADERS/CONTINUATION` block is decoded with `golang.org/x/net/http2/hpack`; padding and embedded priority bytes are removed before decoding. Only pseudo-header names are retained.
3. The observer emits `akamai_text` as `SETTINGS|WINDOW_UPDATE|PRIORITY|PSEUDO_ORDER` and its full MD5 as `akamai_hash`. Standalone PRIORITY frames form the third component; embedded HEADERS priority does not. Missing initial WINDOW_UPDATE is encoded as `00`, and no standalone PRIORITY as `0`. The signature is frozen after the first complete header block. Missing or invalid capture produces an empty signature and an explicit completeness/error flag, never a substitute hash.

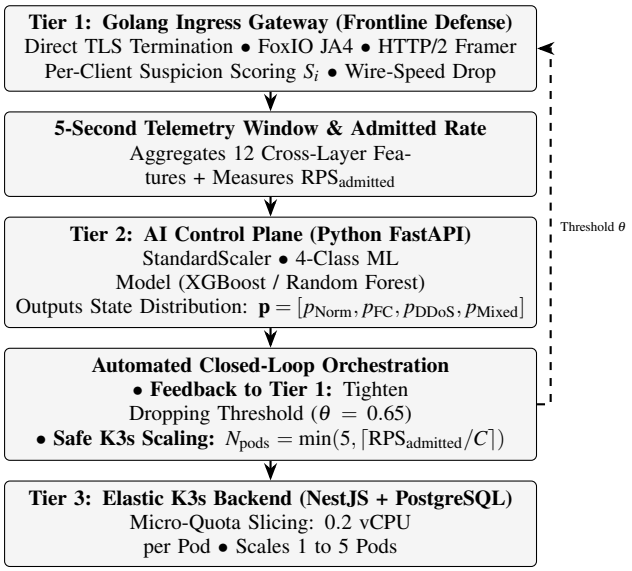


Figure 1: Workflow of the proposed 3-tier sovereign cyber-defense framework.

Regression fixtures supplied from Chrome 156 on BrowserLeaks verify all four JA4 variants and the HTTP/2 signature `1:65536;2:0;4:6291456;6:262144|15663105|0|m,a,s,p`, whose MD5 is `52d84b11737d980aef856699f885ca86`. This is fixture verification, not yet an independent same-connection browser capture. New telemetry uses schema `traffic-lab.v2`; older captures require recollection because missing pseudo-header data cannot be reconstructed from a hash. Fingerprints identify client stack characteristics, not unique users or proof of malicious intent.

To isolate malicious clients during mixed traffic, the gateway computes a real-time **Per-Client Suspicion Score** $S_i \in [0,1]$:

$$S_i = w_1 \mathbb{I}_{(\text{UA} \neq \text{JA4})} + w_2 \mathbb{I}_{(\text{No GREASE})} + w_3 f(\overline{\text{IAT}}_i) + w_4 g(r_i) + w_5 (1 - \mathscr{L}_i) \quad (1)$$

where $\mathbb{I}$ represents indicator functions flagging header-to-fingerprint mismatches and missing GREASE flags, $f(\overline{\text{IAT}}_i)$ penalizes sub-millisecond inter-arrival times, $g(r_i)$ penalizes excessive request frequency, and $\mathscr{L}_i$ is the normalized Markov browsing probability.

When the AI engine detects an attack, the gateway lowers its dropping threshold from $\theta = 0.95$ (normal) to $\theta = 0.65$ (under attack). Sockets with $S_i > \theta$ are terminated immediately with a configurable Time-To-Live (TTL = 60s) to prevent persistent bot flooding.

3.3 The 12 Cross-Layer Domain-Agnostic Features

Every 5 seconds ($\Delta t = 5$ s), the gateway computes 12 statistical features that capture general protocol behavior rather than static URL paths. Table 1 outlines these features.

Table 1: The 12 Cross-Layer Sliding-Window Telemetry Features ($\Delta t = 5$ s) and Hypothesized Baseline Ranges

Layer	Feature Symbol	Operational Definition	Hypothesized: Flash Crowd	Hypothesized: L7 DDoS
L5	$R_{\text{no-grease}}$	Ratio of requests lacking RFC 8701 TLS GREASE flags	Low (< 0.05)	High (0.65–1.00) in scripts/bots
L5	ρ_{JA4}	Ratio of unique JA4 fingerprints to unique client IPs	Diverse (0.15–0.45)	Very Low (< 0.03) as bots share tool
L6.5	M_{cross}	Ratio of User-Agents conflicting with TLS/H2 framing	Near Zero (< 0.02)	High (0.50–1.00) due to spoofing
L6.5	$\bar{S}_{\text{mux}}$	Mean HTTP/2 multiplexed streams per TCP connection	Moderate (4–18)	Extreme (80–500+) in stream floods
L7	$H(\text{IP})$	Shannon entropy of active client IP addresses	High (> 6.5 bits)	Low to Medium (1.5–5.2 bits)
L7	$H(\text{URI})$	Shannon entropy of requested base endpoint URIs	Low (1.2–2.8 bits, event focus)	Very Low (single endpoint) or flat
L7	$H(\text{Query})$	Shannon entropy of URL query parameter strings	Low (< 1.5 bits)	Very High (> 6.0 bits) in CC attacks
L7	R_{static}	Ratio of static assets (.js, .css, images) to total requests	High (0.55–0.80)	Near Zero (< 0.05) for pure API bots
L7	$\bar{r}_{\text{IP}}$	Average request rate per active client IP in window	Human pace (3–14 req)	Aggressive (45–400+ req)
L7	$\overline{\text{IAT}}$	Average per-client inter-arrival time interval	Natural (0.35–1.80 s)	Sub-millisecond (< 0.05 s)
L7	$\mathscr{L}_{\text{Markov}}$	Log-likelihood score of sequential browsing transitions	High (> -1.8, logical flow)	Low (< -5.5, random-walk crawling)
L7	$\bar{T}_{\text{up}}$	Mean backend response latency (dynamic feedback signal)	Low to Moderate (< 0.25 s)	Severe spike (> 1.20 s) under load

4 MACHINE LEARNING TRAINING STRATEGY

4.1 Realistic Dataset Synthesis Without Shortcuts

Machine learning models often suffer from *shortcut learning* (e.g., memorizing that a testing tool like `k6` equals flash crowd while Python `aiohttp` equals DDoS). To eliminate this flaw, our data generation enforces two strict principles:

1. **Real Browser Emulation for Authentic Users:** At least 50% of legitimate user traffic is generated using headless **Playwright (Chromium)** instances. These generate authentic TLS 1.3 handshakes with real GREASE flags, valid browser JA4 fingerprints, and parallel HTTP/2 fetching of static assets.
2. **Cross-Tooling Across All Classes:** Multiple tools are mixed across all categories to prevent tool-signature confounding:
   - *Flash Crowd:* Playwright browsers mixed with randomized `k6` virtual users executing authentic think-times.
   - *L7 DDoS:* Python `aiohttp` (CC cache-busting), `wrk` (HTTP floods), HTTP/2 Rapid Reset scripts, and headless Playwright crawling bots.
3. **Source IP Diversity on LAN:** To prevent Shannon entropy $H(\text{IP})$ from collapsing to zero on a local testbed, load generator nodes utilize IP aliasing (64–128 virtual IPs) and Docker network subnet pools (`172.28.0.0/16`) with the `-local-ips` binding flag.

The planned corpus comprises 2,400 to 3,600 sliding-window samples ($\Delta t = 5$ s), equivalent to 3.5 to 5 hours of continuous multi-scenario execution across 25 independent test sessions.

4.2 Four-Class Labeling and Leak-Free Cross-Validation

Rather than a simplistic binary classification, the AI engine recognizes four distinct macro states:

- **Class 0 (`Normal`):** Baseline daily operations (15–30 req/s).
- **Class 1 (`Flash_Crowd`):** Surges of authentic users (150–350 req/s).
- **Class 2 (`Pure_L7_DDoS`):** High-rate botnet flooding (150–450 req/s).
- **Class 3 (`Mixed_Traffic`):** DDoS attack launched during an active flash crowd (200 req/s authentic +200 req/s botnet traffic).

GroupKFold Cross-Validation: Because adjacent 5-second sliding windows are temporally correlated, standard random k-fold splitting causes severe data leakage. We enforce **GroupKFold** grouped strictly by independent session ID (`session_id`). All windows from a single execution run reside exclusively in either the training or testing partition. Furthermore, **SMOTE** over-sampling is executed strictly inside each training fold, ensuring the holdout test set remains untainted.

Leave-One-Tool-Out (LOTO) Evaluation: To verify genuine generalization, one attack tool (e.g., a slow-rate HTTP POST tool) is withheld entirely from the training folds and evaluated solely during final testing.

4.3 Candidate Model Benchmarking

Five supervised algorithms will be systematically benchmarked:

- **XGBoost (Primary Candidate):** Gradient-boosted decision trees with L_1/L_2 regularization, optimized for tabular data and sub-5 ms inference.
- **Random Forest (RF):** An ensemble of 100 bagged trees with restricted depth (`max_depth=15`) to establish a robust bagging baseline [1].
- **Decision Tree (CART):** An interpretable, rule-based baseline.
- **Multi-Layer Perceptron (MLP):** A fully connected neural network (two hidden layers: 64×32 with ReLU) evaluating non-linear representations.
- **Logistic Regression (LR):** A linear baseline assessing feature separability.

4.4 Closed-Loop Decision Logic and Admitted Scaling

When total traffic exceeds 60 req/s, the orchestrator evaluates the window and triggers automated closed-loop actions:

1. **Normal / Pure Flash Crowd:** The gateway maintains relaxed filtering ($\theta = 0.95$). The orchestrator scales K3s backend pods according to the admitted request volume:

$$N_{\text{pods}} = \min\left(N_{\max}, \left\lceil \frac{\text{RPS}_{\text{admitted}}}{C} \right\rceil\right) \tag{2}$$

where $\text{RPS}_{\text{admitted}}$ is the actual rate of requests forwarded through the gateway, $N_{\max} = 5$ pods, and C is the empirical single-pod saturation capacity (0.2 vCPU) measured during

Week 1 baseline testing (estimated at $C \approx 40$–45 req/s).

2. **Pure DDoS / Mixed Attack:** The orchestrator sends a signed API command to the gateway, tightening the dropping threshold to $\theta = 0.65$. The gateway immediately drops hostile connections with $S_i > 0.65$. Backend pod scaling continues to follow Equation 2 using $\text{RPS}_{\text{admitted}}$. Because bot sockets are severed at the gateway, $\text{RPS}_{\text{admitted}}$ reflects only genuine user demand, completely eliminating EDoS resource waste.
3. **Cooldown and Fail-Safe:** When traffic subsides below 35 req/s for 60 seconds, K3s scales surplus pods back to 1 replica. If the AI engine becomes unreachable (timeout > 500 ms), the gateway activates a local token-bucket fail-safe mode while preserving current pod replicas.

5 EXPERIMENTAL SETUP AND PROJECT ROADMAP

5.1 Micro-Quota Testbed and CPU Isolation

To rigorously evaluate the framework on a local testbed without expensive multi-server cloud clusters, we employ **Kubernetes Micro-Quota Slicing**:

- Each NestJS pod is constrained to `limits.cpu: "200m"` (0.2 vCPU) and `limits.memory: "256Mi"`.
- Due to Node.js's single-threaded event loop, allocating 0.2 vCPU restricts 1 pod's capacity to ~ 45 req/s (latency rises $> 1{,}500$ ms beyond this). When scaled to 5 pods (1.0 vCPU total), latency drops back below 80 ms.
- **CPU Contention Control:** To avoid CPU thrashing on a single physical host, processes are pinned using `cpuset`: Core 0–1 for K3s worker pods, Core 2 for the Golang Ingress Gateway, and Core 3 for the traffic generators.

5.2 Baseline Comparisons and Ablation Study

The proposed defense will be evaluated against two industry baselines:

1. **Baseline 1 (Vanilla CPU HPA):** Standard Kubernetes autoscaling triggered when CPU utilization exceeds 80%.
2. **Baseline 2 (Static IP Rate Limiter):** Gateway rate limiting restricting requests strictly by source IP frequency (> 30 req/s).

An **Ablation Study** will measure classification performance across three feature subsets: (1) Layer-7 behavioral features only, (2) Layer-7 + Layer-5 JA4 features, and (3) the full 12 cross-layer feature set.

5.3 Capstone Implementation Schedule

The project follows a 6-week roadmap with clear role distribution:

- **Week 1 (Oct 19 – Oct 26):** Deploy Go Ingress Gateway with TLS callback extraction; verify FoxIO JA4 and BrowserLeaks-compatible Akamai fingerprints; deploy K3s NestJS backend; benchmark empirical single-pod capacity C. *(Lead: Thien Le & Huy Nguyen).*
- **Week 2 (Oct 27 – Nov 03):** Build Playwright browser generator and cross-tooling scripts; configure IP aliasing pools; capture HTTP/2 frames; collect 3,000 multi-scenario samples. *(Lead: Huy Nguyen & Bao Vo).*
- **Weeks 3–4 (Nov 04 – Nov 18):** Implement leak-free GroupKFold and in-fold SMOTE; train the 5 candidate classifiers; execute LOTO evaluation and ablation studies; code the per-client scorer S_i in Go. *(Lead: Bao Vo & Thien Le).*
- **Week 5 (Nov 19 – Nov 26):** Integrate Python orchestrator with Go Gateway via token-authenticated API; connect K3s scaling client; build Grafana real-time dashboard. *(Lead: All members).*
- **Week 6 (Nov 27 – Dec 04):** Conduct continuous live trial (Flash Crowd $\rightarrow$ Mixed Attack $\rightarrow$ Cooldown); measure Time-to-Mitigate; rehearse the 5-minute defense demo. *(Lead: All members).*

5.4 Planned 5-Minute Live Demonstration Scenario

For the capstone evaluation, the presentation screen will feature a dual display: **Left:** Real-time Grafana Dashboard (showing total RPS, $\text{RPS}_{\text{admitted}}$, 4-class AI probabilities, response latency, and pod replicas); **Right:** Terminal monitoring `kubectl get pods -w` and Go gateway connection drop logs:

1. **Phase 1 (Flash Crowd – Min 1 to 2):** Playwright and `k6` launch authentic student traffic (220 req/s). AI classifies `Flash_Crowd`. The gateway maintains $\theta = 0.95$. K3s scales pods from $1/1$ to $5/5$. Backend response latency stays below 80 ms.
2. **Phase 2 (Mixed Attack – Min 3 to 4):** Botnet scripts launch cache-busting CC attacks during the flash crowd. Total traffic reaches 400 req/s. AI flags `Mixed_Traffic`. Gateway tightens threshold to $\theta = 0.65$ and drops bot sockets. Admitted traffic remains stable at ~ 200 req/s; K3s maintains 5 pods without crashing or suffering EDoS.
3. **Phase 3 (Cooldown – Min 5):** All traffic terminates. After a 60-second cooldown, K3s scales surplus pods back to $1/1$.

6 CONCLUSION

This preliminary report has detailed an autonomous, domain-agnostic cyber-defense framework that eliminates external CDN dependencies through a sovereign Golang Ingress Gateway and machine learning control plane. By combining FoxIO JA4 extraction and Akamai HTTP/2 fingerprinting, and Layer-7 behavioral entropy, the system extracts 12 discriminative features over 5-second sliding windows. The two-tier hierarchical defense combines macro 4-class AI classification with micro per-client suspicion scoring to disentangle mixed attacks during flash crowds. Scaling Kubernetes pods strictly based on admitted legitimate traffic guarantees service availability for authentic users while preventing EDoS. Subsequent phases will execute the empirical benchmarks and live demonstration following the established schedule.

REFERENCES

[1] M. Abdulkadir, M. A. Ahmad, A. A. Aliyu, S. Abdulkadir, M. Ibrahim, A. Abdullahi, and A. M. Ahmed, "Detecting DDoS attacks amidst flash crowds using machine learning," *Bima Journal of Science and Technology*, vol. 9, no. 2A, pp. 23–33, Jun. 2025. [DOI: 10.64290/bimagombe.v9i2A.1061]

[2] J. Althouse, J. Atkinson, and J. Kipp, "JA4+ network fingerprinting suite: JA4, JA4S, JA4H, JA4L, JA4X, JA4SSH," *FoxIO Technical Specification*, 2024. [GitHub: FoxIO-LLC/ja4]

[3] arXiv Security Research Group, "When handshakes tell the truth: Detecting web bad bots via TLS fingerprints," *arXiv preprint arXiv:2602.09606*, Feb. 2026. [arXiv: 2602.09606]

[4] B. A. Muse and S. L. Abebe, "Application layer DDoS attack detection in the presence of flash crowd," *Zede Journal*, vol. 38, no. 1, pp. 1–12, Feb. 2023. [Link: AJOL 242220]

[5] T. Althobaiti, Y. Sanjalawe, and N. Ramzan, "Securing cloud computing from flash crowd attack using ensemble intrusion detection system," *Computer Systems Science and Engineering (CSSE)*, vol. 47, no. 1, pp. 453–469, 2023. [DOI: 10.32604/csse.2023.037484]

[6] S. I. Tinubu, O. A. Ojesanmi, and E. O. Asani, "An intelligent model for DDoS attack detection and flash event management," *International Journal of Distributed Artificial Intelligence (IJDAI)*, vol. 14, no. 1, pp. 1–15, 2022. [DOI: 10.4018/IJDAI.289872]

[7] U. Nag, N. V. Patil, A. Bhardwaj, and C. R. Krishna, "Classification of DDoS attacks and flash events using machine learning: Comparative evaluation on CIC and simulated datasets," in *Proc. IEEE AI Summit*, Nov. 2025, pp. 1–6. [IEEE Xplore: 11411154]

[8] A. M. Biruk, "Application layer DDoS attack detection in the presence of flash crowds," *EEA Journal of Electrical Engineering and Automation*, vol. 2, no. 1, pp. 45–56, 2020. [Link: EEA]

[9] J. David and C. Thomas, "Discriminating flash crowds algorithm using efficient thresholding algorithm," *Journal of Parallel and Distributed Computing*, vol. 152, pp. 79–87, 2021. [DOI: 10.1016/j.jpdc.2021.02.019]

[10] J. B. Gera and P. Bhattacharya, "Detection of spoofed and non-spoofed DDoS attacks and discriminating them from flash crowds," *EURASIP Journal on Information Security*, vol. 2018, no. 1, pp. 1–14, 2018. [DOI: 10.1186/s13635-018-0081-3]

[11] D. Benjamin, "Applying Generate Random Extensions And Sustain Extensibility (GREASE) to TLS extensibility," *RFC 8701*, IETF, Feb. 2020. [DOI: 10.17487/RFC8701]

[12] Rancher Labs, "K3s: The lightweight Kubernetes distribution," *Rancher Documentation*, 2024. [Online Docs]

[13] O. Segal, A. Fridman, and E. Shuster, "Passive fingerprinting of HTTP/2 clients," *Akamai White Paper, Black Hat Europe*, 2017. [White Paper]

[14] BrowserLeaks, "TLS client test and HTTP/2 fingerprint," accessed Oct. 23, 2026. [TLS Test] [HTTP/2 JSON]